KOÇ UNIVERSITY

# COMP 350
# Introduction to DevOps

## Lecture 6
## Shell Scripting

Hakan Ayral

# A First Script

- Create a text file (first.sh) as follows:

```
#!/bin/sh
# This is a comment!
echo Hello World        # This is a comment, too!
```

- The first line tells Unix that the file is to be executed by **/bin/sh**
  - **/bin/sh** is normally a symbolic link to **bash**
- **chmod 755 first.sh** to make the text file executable

```
$ chmod 755 first.sh
$ ./first.sh
Hello World
$
```

# What would be the output?

```
#!/bin/sh
# This is a comment!
echo "Hello      World"       # This is a comment, too!
echo "Hello World"
echo "Hello * World"
echo Hello * World
echo Hello      World
echo "Hello" World
echo Hello "     " World
echo "Hello "*" World"
echo `hello` world
echo 'hello' world
```

```
hakan@LT-PW06ZDBZ:~$ ./test.sh
Hello      World
Hello World
Hello * World
Hello Lecture Notes MC-Projects test.sh test.txt World
Hello World
Hello World
Hello      World
Hello * World
./test.sh: 1: hello: not found
world
hello world
hakan@LT-PW06ZDBZ:~$
```

# Variables

- VAR=value
  - there must be no spaces around the "=" sign: **VAR=value** works; **VAR = value** doesn't work.

```
#!/bin/sh
MY_MESSAGE="Hello World"
echo $MY_MESSAGE
```

- The shell does not care about types of variables
  - They are all stored as strings, but routines which expect a number can treat them as such.

```
$ x="hello"
$ expr $x + 1
expr: non-numeric argument
$
```

# Read

```
#!/bin/sh
echo What is your name?
read MY_NAME
echo "Hello $MY_NAME - hope you're well."
```

- The whole idea of shell scripts is automation
  - read interrupts the execution and waits for interactive input from the user, which breaks automation (execution without user intervention)
  - so don’t use read unless you have to.
- Variables in the Bourne shell do not have to be declared
  - But if you try to read an undeclared variable, the result is the empty string.
    - You get no warnings or errors.
    - This can cause some subtle bugs (e.g. When you misspell var names)

# Export

- **export** has a fundamental effect on the scope of variables
- Create a small shell script, **myvar2.sh**:

```
#!/bin/sh
echo "MYVAR is: $MYVAR"
MYVAR="hi there"
echo "MYVAR is: $MYVAR"
```

- Run the script:

```
$ ./myvar2.sh
MYVAR is:
MYVAR is: hi there
```

- MYVAR wasn't set to any value, so it was blank. Then we give it a value, and it has the expected result.
- Now run:

```
$ MYVAR=hello
$ ./myvar2.sh
MYVAR is:
MYVAR is: hi there
```

- It's still not been set?!

# Export

- When you call **myvar2.sh** from your interactive shell, a new shell is spawned to run the script.
- We need to **export** the variable for it to be inherited by another program - including a shell script:

```
$ export MYVAR
$ ./myvar2.sh
MYVAR is: hello
MYVAR is: hi there
```

- Script is changing the value of **MYVAR**. But there is no way that this will be passed back to your interactive shell.
- Try reading the value of **MYVAR**:

```
$ echo $MYVAR
hello
$
```

- Once the shell script exits, its environment is destroyed. But **MYVAR** keeps its original value of **hello** within your interactive shell.

# Export / source

- In order to receive environment changes back from the script, we must *source* the script
  - this runs the script within our own interactive shell, instead of spawning another shell to run it.
- We can source a script via the "." (dot) command:
- Change has made it out into our shell again
  - This is how your **.profile** or **.bash_profile** file works, for example.
- We no longer need to **export MYVAR**.
- String interpolation in shell:

```
$ MYVAR=hello
$ echo $MYVAR
hello
$ . ./myvar2.sh
MYVAR is: hello
MYVAR is: hi there
$ echo $MYVAR
hi there
```

```
#!/bin/sh
echo "What is your name?"
read USER_NAME
echo "Hello $USER_NAME"
echo "I will create you a file called $USER_NAME_file"
touch $USER_NAME_file
```

```
#!/bin/sh
echo "What is your name?"
read USER_NAME
echo "Hello $USER_NAME"
echo "I will create you a file called ${USER_NAME}_file"
touch "${USER_NAME}_file"
```

# For Loops

- **for** loops iterate through a set of values until the list is exhausted
- Another example:

```
#!/bin/sh
for i in 1 2 3 4 5
do
  echo "Looping ... number $i"
done
```

```
#!/bin/sh
for i in hello 1 * 2 goodbye
do
  echo "Looping ... i is set to $i"
done
```



```
hakan@LT-PW06ZDBZ:~$ ./test.sh
Looping ... i is set to hello
Looping ... i is set to 1
Looping ... i is set to Lecture Notes
Looping ... i is set to MC-Projects
Looping ... i is set to test.sh
Looping ... i is set to test.txt
Looping ... i is set to 2
Looping ... i is set to goodbye
hakan@LT-PW06ZDBZ:~$
```

# While Loops

- echo and read statements will run indefinitely until you type "bye" when prompted.
- colon (:) always evaluates to true

```
#!/bin/sh
while :
do
  echo "Please type something in (^C to quit)"
  read INPUT_STRING
  echo "You typed: $INPUT_STRING"
done
```

- A practical example

```
for runlevel in 0 1 2 3 4 5 6 S
do
  mkdir rc${runlevel}.d
done
```



```
mkdir rc{0,1,2,3,4,5,6,S}.d
```

```
#!/bin/sh
INPUT_STRING=hello
while [ "$INPUT_STRING" != "bye" ]
do
  echo "Please type something in (bye to quit)"
  read INPUT_STRING
  echo "You typed: $INPUT_STRING"
done
```

# Another trick



```
$ cd /
$ ls -ld {,usr,usr/local}/{bin,sbin,lib}
drwxr-xr-x    2 root     root         4096 Oct 26 01:00 /bin
drwxr-xr-x    6 root     root         4096 Jan 16 17:09 /lib
drwxr-xr-x    2 root     root         4096 Oct 27 00:02 /sbin
drwxr-xr-x    2 root     root        40960 Jan 16 19:35 usr/bin
drwxr-xr-x   83 root     root        49152 Jan 16 17:23 usr/lib
drwxr-xr-x    2 root     root         4096 Jan 16 22:22 usr/local/bin
drwxr-xr-x    3 root     root         4096 Jan 16 19:17 usr/local/lib
drwxr-xr-x    2 root     root         4096 Dec 28 00:44 usr/local/sbin
drwxr-xr-x    2 root     root         8192 Dec 27 02:10 usr/sbin
```

# Testing Conditions

- **test** is not often called directly. **test** is more frequently called as **[** *(← yes, the left square bracket)*
  - **[** is a symbolic link to **test**, just to make shell scripts more readable.
  - Alternatively it is also normally a shell builtin → shell itself will interpret **[** as meaning **test**, even if you don't have the symbolic link

```
hakan@LT-PW06ZDBZ:~$ type [
[ is a shell builtin
hakan@LT-PW06ZDBZ:~$ which [
/usr/bin/[
```

- **[** is actually a program, just like **ls** and other programs, so it must be surrounded by spaces
  - For example `if [$foo = "bar" ]` will not work because it is interpreted as **`if test$foo = "bar" ]`**
  - This one works: `if [ "$foo" = "bar" ]`
  - Some shells also accept "**==**" for string comparison; this is not portable, a single "**=**" should be used for strings, or "**-eq**" for integers.
- **test** is not used on its own it is used indirectly via the **if** and **while** statements

# Test

- The syntax for if...then...else... is:

```
if [ ... ]
then
  # if-code
else
  # else-code
fi
```

- the "**if [ ... ]**" and the "**then**" commands must be on different lines.
- Alternatively, the semicolon "**;**" can separate them:

```
if [ ... ]; then
  # do something
fi
```

- Test.sh demo here with X =
  - -1, 0, 1
  - hello, bye
  - test.sh

- Tip:
  - You can use the semicolon **;** to join two lines together which should be separate lines
  - **** is opposite: it tells the shell that this is not the end of the line, but that the following line should be treated as part of the current line

# Test Expressions

```
test EXPRESSION
test
[ EXPRESSION ]
[ ]
[ OPTION
```

```
( EXPRESSION )
       EXPRESSION is true

! EXPRESSION
       EXPRESSION is false

EXPRESSION1 -a EXPRESSION2
       both EXPRESSION1 and EXPRESSION2 are true

EXPRESSION1 -o EXPRESSION2
       either EXPRESSION1 or EXPRESSION2 is true

-n STRING
       the length of STRING is nonzero

STRING equivalent to -n STRING

-z STRING
       the length of STRING is zero

STRING1 = STRING2
       the strings are equal

STRING1 != STRING2
       the strings are not equal

STRING1 > STRING2
       STRING1 is greater than STRING2 in the current locale

STRING1 < STRING2
       STRING1 is less than STRING2 in the current locale
```

```
INTEGER1 -eq INTEGER2
       INTEGER1 is equal to INTEGER2

INTEGER1 -ge INTEGER2
       INTEGER1 is greater than or equal to INTEGER2

INTEGER1 -gt INTEGER2
       INTEGER1 is greater than INTEGER2

INTEGER1 -le INTEGER2
       INTEGER1 is less than or equal to INTEGER2

INTEGER1 -lt INTEGER2
       INTEGER1 is less than INTEGER2

INTEGER1 -ne INTEGER2
       INTEGER1 is not equal to INTEGER2

FILE1 -ef FILE2
       FILE1 and FILE2 have the same device and inode numbers

FILE1 -nt FILE2
       FILE1 is newer (modification date) than FILE2

FILE1 -ot FILE2
       FILE1 is older than FILE2
```

```
-b FILE
       FILE exists and is block special

-c FILE
       FILE exists and is character special

-d FILE
       FILE exists and is a directory

-e FILE
       FILE exists

-f FILE
       FILE exists and is a regular file

-g FILE
       FILE exists and is set-group-ID

-G FILE
       FILE exists and is owned by the effective group ID

-h FILE
       FILE exists and is a symbolic link (same as -L)

-k FILE
       FILE exists and has its sticky bit set

-L FILE
       FILE exists and is a symbolic link (same as -h)

-N FILE
       FILE exists and has been modified since it was last read

-O FILE
       FILE exists and is owned by the effective user ID

-p FILE
       FILE exists and is a named pipe

-r FILE
       FILE exists and the user has read access

-s FILE
       FILE exists and has a size greater than zero

-S FILE
       FILE exists and is a socket

-t FD  file descriptor FD is opened on a terminal

-u FILE
       FILE exists and its set-user-ID bit is set

-w FILE
       FILE exists and the user has write access

-x FILE
       FILE exists and the user has execute (or search) access
```

# Test

- There is a simpler way of writing **if** statements: **&&** and **||** commands give code to run if the result is true, or false, respectively.
  - **Caution: here && and || does NOT mean logical and, logical or**

```
#!/bin/sh
[ $X -ne 0 ] && echo "X isn't zero" || echo "X is zero"
[ -f $X ] && echo "X is a file" || echo "X is not a file"
[ -n $X ] && echo "X is of non-zero length" || \
      echo "X is of zero length"
```

# Guessing game

- **$?** is a special variable. It contains the exit value of the last run command.
- **>/dev/null 2>&1** directs any output or errors to the special "null" device, instead of going to the user's screen
- **grep [0-9]** finds lines of text which contain at least one digit (0-9) and possibly other characters
  - the caret (**^**) in **grep [^0-9]** finds only those lines which don't consist only of numbers

```
echo -en "Please guess the magic number: "
read X
echo $X | grep "[^0-9]" > /dev/null 2>&1
if [ "$?" -eq "0" ]; then
  # If the grep found something other than 0-9
  # then it's not an integer.
  echo "Sorry, wanted a number"
else
  # The grep found only 0-9, so it's an integer.
  # We can safely do a test on it.
  if [ "$X" -eq "7" ]; then
    echo "You entered the magic number!"
  fi
fi
```

# Test in while loop

- This script will keep asking for input until you hit RETURN (X is zero length).
- An easy mistake to make - if you miss out the quotes around $X in the **while [ -n "$X" ]**, there is nothing to test if $X is empty.
  - **while [ -n ]** is **invalid** syntax, and will fail.
  - **while [ -n "" ]** is valid syntax, so to allow for the variable being empty, you do need to include the quotes.

```
#!/bin/sh
X=0
while [ -n "$X" ]
do
  echo "Enter some text (RETURN to quit)"
  read X
  echo "You said: $X"
done
```

# Variables (continued)

- For variables in Bash, there are two well-known good practices.
  1. Always put curly brackets {} around the variable name.
  2. Always put double quotes "" around the variable.

- Therefore, a variable should look like this: "`${variable_name}`“
- Example: Create a directory named “tips” and a file inside named “filename with spaces”

```
for filename in $(ls tips); do
  echo Found: ${filename}
  cat tips/${filename}
done
```

```
Found: Filename
cat: tips/Filename: No such file or directory
Found: with
cat: tips/with: No such file or directory
Found: spaces
cat: tips/spaces: No such file or directory
```

```
for filename in "$(ls tips)"; do
  echo Found: "${filename}"
  cat tips/"${filename}"
done
```

```
Found: Filename with spaces
File content
```

# Variables (continued)

- The variable **$0** is the *basename* of the program as it was called.
- **$1 .. $9** are the first 9 additional parameters the script was called with.
- The variable **$@** is all parameters **$1 .. *(up to last param)***.
- The variable **$***, is similar, but does not preserve any whitespace, and quoting
  - "File with spaces" becomes "File" "with" "spaces".
- As a general rule, use **$@** and avoid **$***.
- **$#** is the number of parameters the script was called with.
- **$?** contains the exit value of the last run command.
- **$$** variable is the PID of the currently running shell.
  - This can be useful for creating temporary files, such as **/tmp/my-script.$$** which is useful if many instances of the script could be run at the same time, and they all need their own temporary files.
- The **$!** variable is the PID of the last run background process.
  - This is useful to keep track of the process as it gets on with its job.

# Variables (continued)

```
foo=sun
echo $fooshine     # $fooshine is undefined
echo ${foo}shine   # displays the word "sunshine"
```

- Curly braces have another powerful use. We can deal with issues of variables being **undefined** or **null**.

don't add linebreak

```
#!/bin/sh
echo -en "What is your name [ `whoami` ] "
read myname
if [ -z "$myname" ]; then
  myname=`whoami`
fi
echo "Your name is : $myname"
```

```
echo -en "What is your name [ `whoami` ] "
read myname
echo "Your name is : ${myname:-`whoami`}"
```

- "**:-**", sets **default value if** variable is **undefined**:
- There is another syntax, "**:=**", which sets the variable to the default if it is undefined:

```
echo "Your name is : ${myname:=John Doe}"
```

# Functions

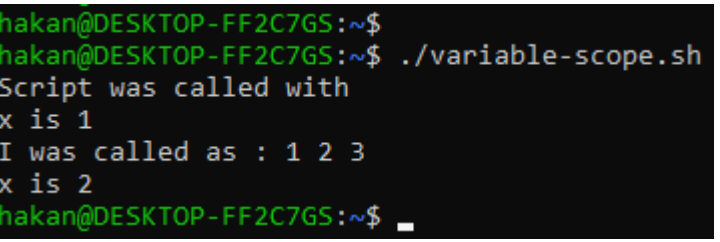
```
<function-name>()
{
    <shell-command>
    …
}
```

```
 GNU nano 7.2   variable-scope.sh
#!/bin/sh

myfunc()
{
  echo "I was called as : $@"
  x=2
}

### Main script starts here

echo "Script was called with $@"
x=1
echo "x is $x"
myfunc 1 2 3
echo "x is $x"
```




- There is no variable scope or local variables (i.e. all variables are global!)
  - except (**$1**, **$2**, **$@**, etc.)
  - Functions cannot change the values they have been called with
- Similar to calling another script with command line arguments
  - You do not declare parameters they are passed with **$1**, **$2, $3, …**

# Functions

```
#!/bin/sh
# A simple script with a function...

add_a_user()
{
  USER=$1
  PASSWORD=$2
  shift; shift;
  # Having shifted twice, the rest is now comments ...
  COMMENTS=$@
  echo "Adding user $USER ..."
  echo useradd -c "$COMMENTS" $USER
  echo passwd $USER $PASSWORD
  echo "Added user $USER ($COMMENTS) with pass $PASSWORD"
}

###
# Main body of script starts here
###
echo "Start of script..."
add_a_user bob letmein Bob Holness the presenter
add_a_user fred badpassword Fred Durst the singer
add_a_user bilko worsepassword Sgt. Bilko the role model
echo "End of script..."
```

Move parameters to the left like dequeue or pop
(*i.e. $1 gets the value of $2, $2 gets the value of $3, etc..*)

# Quick reference

| Command | Description | Example |
|---|---|---|
| & | Run the previous command in the background | `ls &` |
| && | Logical AND | `if [ "$foo" -ge "0" ] && [ "$foo" -le "9"]` |
| \|\| | Logical OR | `if [ "$foo" -lt "0" ] \|\| [ "$foo" -gt "9" ]` |
| ^ | Start of line | `grep "^foo"` |
| $ | End of line | `grep "foo$"` |
| = | String equality (cf. -eq) | `if [ "$foo" = "bar" ]` |
| ! | Logical NOT | `if [ "$foo" != "bar" ]` |
| $$ | PID of current shell | `echo "my PID = $$"` |
| $! | PID of last background command | `ls & echo "PID of ls = $!"` |
| $? | exit status of last command | `ls ; echo "ls returned code $?"` |
| $0 | Name of current command (as called) | `echo "I am $0"` |
| $1 | Name of current command's first parameter | `echo "My first argument is $1"` |
| $9 | Name of current command's ninth parameter | `echo "My ninth argument is $9"` |
| $@ | All of current command's parameters (preserving whitespace and quoting) | `echo "My arguments are $@"` |
| $* | All of current command's parameters (not preserving whitespace and quoting) | `echo "My arguments are $*"` |

| | | |
|---|---|---|
| -eq | Numeric Equality | `if [ "$foo" -eq "9" ]` |
| -ne | Numeric Inquality | `if [ "$foo" -ne "9" ]` |
| -lt | Less Than | `if [ "$foo" -lt "9" ]` |
| -le | Less Than or Equal | `if [ "$foo" -le "9" ]` |
| -gt | Greater Than | `if [ "$foo" -gt "9" ]` |
| -ge | Greater Than or Equal | `if [ "$foo" -ge "9" ]` |
| -z | String is zero length | `if [ -z "$foo" ]` |
| -n | String is not zero length | `if [ -n "$foo" ]` |
| -nt | Newer Than | `if [ "$file1" -nt "$file2" ]` |
| -d | Is a Directory | `if [ -d /bin ]` |
| -f | Is a File | `if [ -f /bin/ls ]` |
| -r | Is a readable file | `if [ -r /bin/ls ]` |
| -w | Is a writable file | `if [ -w /bin/ls ]` |
| -x | Is an executable file | `if [ -x /bin/ls ]` |
| ( ... ) | Function definition | `function myfunc() { echo hello }` |

# ` ← backtick

- Capture the output of a program to a variable:

```
hakan@DESKTOP-FF2C7GS:~$
hakan@DESKTOP-FF2C7GS:~$ whoami
hakan
hakan@DESKTOP-FF2C7GS:~$ NAME=`whoami`
hakan@DESKTOP-FF2C7GS:~$ echo $NAME
hakan
hakan@DESKTOP-FF2C7GS:~$
```

- There is an alternative way:

```
hakan@DESKTOP-FF2C7GS:~$
hakan@DESKTOP-FF2C7GS:~$ NAME2=$(whoami)
hakan@DESKTOP-FF2C7GS:~$ echo $NAME2
hakan
hakan@DESKTOP-FF2C7GS:~$
```

# Examples

- Count the files

```
  GNU nano 7.2                 file-count.sh
#!/bin/bash

# Initialize counter variable
file_count=0

# Loop through all files in directory and count them
for file in *; do
 if [ -f "$file" ]; then
  file_count=$((file_count+1))
  echo "$file"
 fi
done

# Display a message indicating the total number of files
echo "There are $file_count files in this directory."
```

- Check if all the codes in each subdirectory compile without error using make:

```
for d in */ ; do
 cd $d
 make clean
 make all
 echo $d , $? $([ $? -eq 0 ] && echo "OK" || echo "FAIL")  >>  ../test-results.txt
 cd ..
done
```

cd into each first level subdirectory and compile all .c files there:

```
for d in */ ; do
    cd "$d"
    for f in *.c ; do
       gcc "$f" > /dev/null 2>&1
       echo "$d" "$f" $?
    done
    cd ..
done
```

Execute a given command for all files containing a specific string:

```
grep "Label/youtube" * -l | xargs  -I % mv "%" "youtube"
```

Remove prefix from github classroom team name:

```
find . -type d -maxdepth 1 -exec bash -c 'mv ${0} ${0/comp-304-spring2025-project-1-shell-/}' {} \;
```

JULIA EVANS
@b0rk

# brackets cheat sheet

## shell scripts have a lot of brackets




## (cd ~/music; pwd)

(...) runs commands in a subshell.

## { cd ~/music; pwd; }

{...} groups commands. runs in the same process.

## x=$((2+2))

$(()) does arithmetic

## if [ ... ]

/usr/bin/[ is a program that evaluates statements

## a{.png,.svg}

this expands to a.png a.svg
it's called "brace expansion"

## if [[ ... ]]

[[ is bash syntax. it's more powerful than [

## VAR=$(cat file.txt)

$(COMMAND) is equal to COMMAND's stdout

## x=(1 2 3)

x=(...) creates an array

## <(COMMAND)

"process substitution": an alternative to pipes

## ${var//search/replace}

see page 21 for more about ${...}!